



SOFTWARE DEVELOPMENT

MERN STACK DEVELOPMENT REPORT

WEEK 2 FOCUS

CSS Grid, Flexbox & Responsive Layout

Muneeb Ur Rehman

MERN STACK INTERN

Objective

The objective of this task was to build a responsive admin dashboard using modern CSS techniques. The main focus was on understanding how **CSS Grid** and **Flexbox** work together to create professional layouts. Along with layout design, the task also included learning CSS Variables, responsive design, overflow handling, BEM naming conventions, transitions, hover effects, and media queries.

What I Learned

1. CSS Variables (:root)

I learned how to store colors and reusable values inside the `:root` selector using CSS Variables.

Example:

```
:root{
  --clr-primary:#3b82f6;
}
```

Instead of writing the same color multiple times, I can simply use:

```
color: var(--clr-primary);
```

This makes the code cleaner and easier to maintain. If I want to change the primary color later, I only need to update it once.

2. CSS Grid

This task helped me understand that **Grid is mainly used for page layout**.

The dashboard is divided into two columns:

- Sidebar
- Main Content

and two rows:

- Header
- Content Area

Example:

```
grid-template-columns:260px 1fr;
grid-template-rows:70px 1fr;
```

I learned that:

- `260px` creates a fixed-width sidebar.
 - `1fr` means "use all the remaining available space."
-

3. Understanding `fr`

Initially I was confused about the meaning of `1fr`.

I learned that `fr` means **fraction**.

For example:

```
grid-template-columns:1fr 1fr;
```

means both columns share the available space equally.

While

```
grid-template-columns:2fr 1fr;
```

means the first column gets twice the space of the second.

4. Understanding Grid Lines

One of the most difficult concepts for me was:

```
grid-column:1/2;  
grid-row:1/3;
```

At first I thought these numbers represented column numbers.

After learning, I understood that Grid positions items using **grid lines (boundaries)** instead of column numbers.

For example:

```
grid-column:1/2;
```

means:

- Start from grid line 1
- End at grid line 2

which places the element inside the first column.

Similarly,

```
grid-row:1/3;
```

starts at row line 1 and ends at row line 3, causing the sidebar to span across both rows.

This was one of the biggest learning points of this task.

5. Flexbox

After Grid placed each section of the page, Flexbox was used inside those sections.

Example:

```
.sidebar{
  display:flex;
  flex-direction:column;
}
```

I learned that:

- Grid decides **where the sidebar is placed**.
- Flexbox decides **how the items inside the sidebar are arranged**.

The same concept was also used in the header.

6. Overflow

I also learned the difference between

```
overflow:auto;
```

and

```
overflow:scroll;
```

Previously I thought both were the same.

Now I understand:

- `overflow:auto` only shows a scrollbar when content exceeds the container.
- `overflow:scroll` always shows the scrollbar even if there is nothing to scroll.

This dashboard uses

```
overflow-y:auto;
```

inside the main content so only the content area scrolls while the sidebar and header remain fixed.

7. Viewport Height

I learned the purpose of

```
height:100vh;
```

It makes the dashboard occupy the entire browser height, creating a full-screen layout.

8. BEM Naming Convention

This project also improved my understanding of BEM.

Examples:

```
.sidebar
.sidebar__nav
.sidebar__link
.sidebar__link--active
```

I learned that:

- Block → sidebar
- Element → sidebar__link
- Modifier → sidebar__link--active

This naming convention makes CSS much easier to organize and maintain.

9. Responsive Design

I learned how Media Queries allow the layout to adapt to different screen sizes.

Example:

```
@media (max-width:850px)
```

On smaller screens:

- The sidebar moves below the header.
- The layout changes from two columns into one column.
- Padding becomes smaller.
- Normal page scrolling is enabled.

This makes the dashboard mobile-friendly.

10. Auto Responsive Grid

I learned a new Grid property:

```
grid-template-columns:  
repeat(auto-fit,minmax(240px,1fr));
```

This automatically creates as many widget columns as possible depending on the available screen width.

As the screen becomes smaller, the widgets automatically move to the next row without requiring additional media queries.

Challenges I Faced

During this task I faced several conceptual challenges rather than coding errors.

The biggest difficulty was understanding how CSS Grid places elements using **grid lines instead of column numbers**. Initially, properties like `grid-column: 1 / 2` and `grid-row: 1 / 3` were confusing because I assumed the numbers referred directly to columns and rows. After learning that these values represent the starting and ending grid lines, the concept became much clearer.

I also struggled to understand the meaning of `1fr`. At first, I thought it was another measurement unit similar to pixels. Through real-life examples, I learned that `fr` represents a fraction of the remaining available space, which makes layouts responsive.

Another area of confusion was the difference between `overflow: auto` and `overflow: scroll`. I originally believed both properties behaved the same. After experimenting with them, I understood that `auto` only displays scrollbars when necessary, while `scroll` always displays them.

Questions I Asked During This Task

While completing this project, I asked several questions to strengthen my understanding instead of simply copying the code.

Some of the questions included:

- What exactly does `1fr` mean?
- Why does CSS Grid use grid lines instead of column numbers?
- Why does `grid-column: 1 / 2` represent the first column?
- Why does `grid-row: 1 / 3` span two rows instead of three?
- What is the practical difference between `overflow: auto` and `overflow: scroll`?
- When should Grid be used instead of Flexbox?
- How do Grid and Flexbox work together in a real dashboard?